DATA ENGINEERING COURSE · PHASE 1: PROJECT PROPOSAL & ENVIRONMENT SETUP

Chicago Bike-Share Analytics Pipeline

An orchestrated pipeline that loads Divvy bike-share trips into PostgreSQL and serves a ridership dashboard.

1. Team Members

Full name	Student ID	Assigned role
Mohamed Ashraf Gamal	692300392	**Team lead & infrastructure**: Docker Compose, PostgreSQL, pgAdmin, repository
Ahmed Walid	692300197	**Ingestion engineer**: Python extraction, cleaning and loading scripts
Khaled Hossam	692400463	**Database & data quality**: table schemas (DDL), quality checks, analytics views
Mazen Walid	692300202	**Orchestration & dashboard**: Kestra flows, scheduling/backfill, dashboard

Roles show who leads each area. Every member runs the full environment on their own laptop and can explain the whole workflow.

2. Dataset

Name	Divvy Trip Data: every ride taken on Chicago's public bike-share system
Link	https://divvy-tripdata.s3.amazonaws.com/index.html
Publisher	Divvy (operated by Lyft for the City of Chicago), under the Divvy Data License Agreement, free for public use
Format	CSV, one zipped file per month (`YYYYMM-divvy-tripdata.zip`), downloadable without an account
Rows	**137,787 rows in the January 2026 file alone** (the smallest month of 2026). Summer files are 5–6× larger by file size. One row = one ride.
Columns	13 (2 timestamps, 4 numeric, 2 categorical, 5 text identifiers/names)
Project scope	The 2026 monthly files (January–October published so far), loaded one month at a time

Why this dataset

- **Meets the brief:** public CSV, far more than 10,000 rows, and it has dates, numbers and categories. It is not the NYC Taxi dataset.
- **Fits the pipeline:** a new file is published every month, so Kestra has a real reason to run on a schedule and to backfill past months.
- **Clean but realistic:** consistent 13-column schema and no duplicate ride IDs, with a few genuine quality issues to handle (below).

Columns and data types (profiled from the January 2026 file)

Column	Data type	Missing	Description
`ride_id`	text	0%	Unique ride identifier
`rideable_type`	category	0%	`electric_bike` / `classic_bike`
`started_at`	timestamp	0%	Ride start date and time
`ended_at`	timestamp	0%	Ride end date and time
`start_station_name`	text	17.8%	Start station name (1,139 distinct)
`start_station_id`	text	17.8%	Start station code
`end_station_name`	text	20.0%	End station name
`end_station_id`	text	20.0%	End station code
`start_lat`, `start_lng`	decimal	0%	Start coordinates
`end_lat`, `end_lng`	decimal	0.1%	End coordinates
`member_casual`	category	0%	`member` (subscriber) / `casual` (pay per ride)

Quality issues found: station name/ID is empty for about 18–20% of rides (bikes left away from a dock); 3,846 rides last under one minute and 191 last over 24 hours; the CSV inside the January 2026 zip carries a 2025 file name. Our cleaning step will handle each of these.

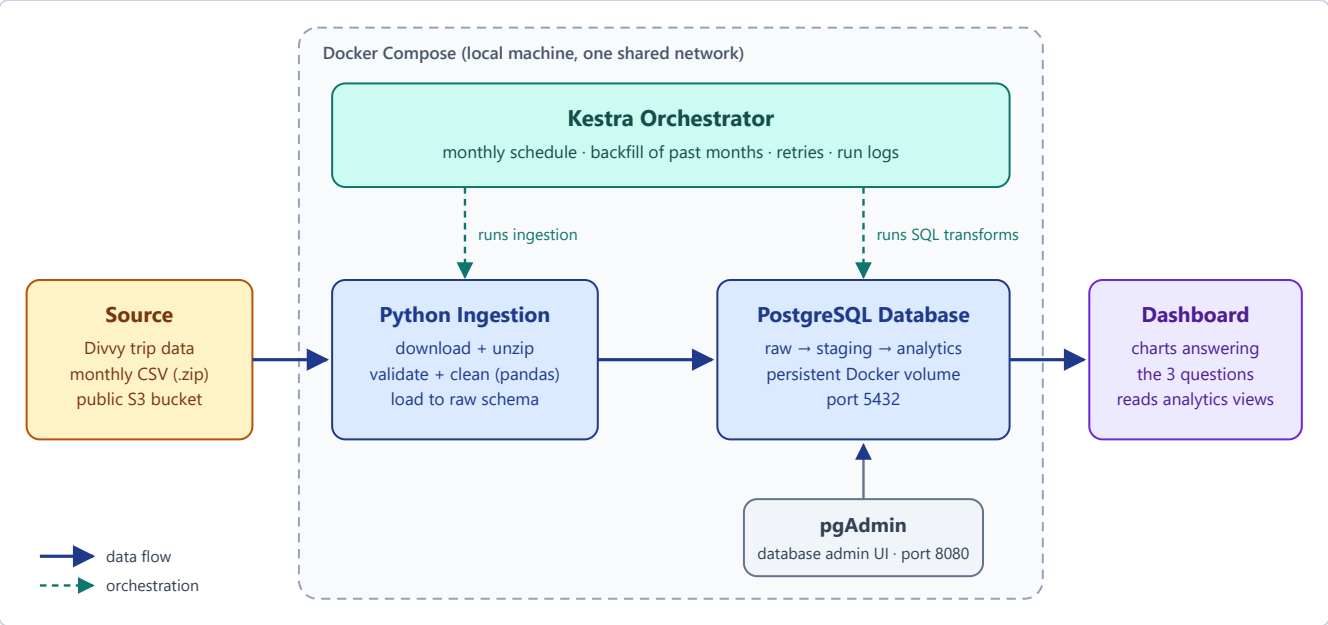
3. Analytical Questions

> **Q1. When do members and casual riders use the system?** How does the number of rides per hour of day differ between members and casual riders, on weekdays versus weekends?
>
> Uses: `started_at`, `member_casual`. Shows commuter peaks versus leisure use.

> **Q2. Which stations are busiest, and which are out of balance?** Each month, which 10 stations have the most ride starts, and which stations have the largest gap between departures and arrivals?
>
> Uses: `start_station_*`, `end_station_*`, `started_at`. Shows where bikes must be moved back.

> **Q3. How do ride length and bike choice change through the year?** How do average ride duration and distance differ between electric and classic bikes and between rider types, month by month?
>
> Uses: `started_at`, `ended_at`, coordinates, `rideable_type`, `member_casual`. Shows seasonality.

4. Proposed Architecture



Data path: [Source] → [Python Ingestion] → [PostgreSQL Database] → [Dashboard], with the [Kestra Orchestrator] scheduling each step.

- **Source:** monthly zipped CSV files on Divvy's public S3 bucket.
- **Python ingestion:** downloads one month, checks the columns, cleans with pandas and loads into PostgreSQL.
- **PostgreSQL:** three schemas: `raw` (as downloaded), `staging` (cleaned), `analytics` (dashboard tables). Data sits on a persistent Docker volume.
- **Kestra:** runs the ingestion every month, backfills earlier months, retries failures and then runs the SQL transforms.
- **Dashboard:** reads the `analytics` views and shows one chart group per question.
- **Phase 1 environment:** PostgreSQL 16 and pgAdmin 4 run from one `docker-compose.yml` on a shared Docker network; pgAdmin reaches the database by the service name `postgres` on port 5432.

5. Expected Tables and Columns

Table	Purpose	Columns
`raw.trips`	Rides exactly as published	The 13 source columns + `source_file`, `loaded_at`
`staging.trips_clean`	Cleaned, typed rides	`ride_id` (PK), `rideable_type`, `member_casual`, `started_at`, `ended_at`, `ride_date`, `ride_hour`, `day_of_week`, `is_weekend`, `duration_minutes`, `distance_km`, `start_station_id`, `end_station_id`
`analytics.dim_station`	One row per station	`station_id` (PK), `station_name`, `latitude`, `longitude`
`analytics.dim_date`	One row per calendar day	`date_key` (PK), `year`, `month`, `month_name`, `day_of_week`, `day_name`, `is_weekend`
`analytics.fact_trips`	One row per valid ride	`ride_id` (PK), `date_key` (FK), `ride_hour`, `start_station_id` (FK), `end_station_id` (FK), `rideable_type`, `member_casual`, `duration_minutes`, `distance_km`